

TITRE PROFESSIONNEL
CONCEPTEUR DÉVELOPPEUR D'APPLICATIONS
Niveau 6 (Cadre national des certifications)
RNCP 37873 — Code titre TP-01281

Dossier de projet

Solage

Réseau social de microblogging — application web sécurisée multicouche

Candidat :

Adam COURTARO

Session :

Projet :

Réalisé en formation

Centre :

Dossier rendant compte de la mise en œuvre des onze compétences professionnelles du titre, réparties sur les trois certificats de compétences professionnelles (CCP).

Document de 40 à 60 pages hors page de garde, sommaire et annexes — schémas et illustrations compris.

Table des matières

1	Liste des compétences mises en œuvre	1
2	Expression des besoins	3
2.1	Contexte et problème adressé	3
2.2	Objectifs du projet	3
2.3	Périmètre et limites	4
2.4	Acteurs et cas d’usage	4
2.5	Contraintes réglementaires et qualité	4
3	Environnement technique	6
3.1	Vue d’ensemble de la pile technique	6
3.2	Justification des choix techniques	6
3.3	Poste de travail et outils	7
3.4	Conteneurisation (Docker)	7
4	Réalisations	9
4.1	Architecture logicielle multicouche	9
4.1.1	Règles de séparation des couches	9
4.1.2	Rôle de sécurité de chaque couche (DICP)	9
4.1.3	Patrons mis en œuvre	10
4.2	Gestion de projet	10
4.3	Maquettes et enchaînement des écrans	11
4.4	Conception de la base de données	12
4.4.1	Modèle conceptuel des données (MCD)	12
4.4.2	Modèle logique / physique (MLD, MPD)	12
4.4.3	Évolution du schéma : migrations idempotentes	14
4.5	Diagrammes UML	15
4.5.1	Diagramme de cas d’utilisation	15
4.5.2	Diagramme de séquence : « Publier un message »	15
4.5.3	Diagramme de classes (couche modèle)	15
4.6	Composants d’interface utilisateur	15
4.7	Composants métier	18
4.7.1	Contrôle d’accès : la parade à l’IDOR	18
4.7.2	Authentification <i>vs</i> autorisation	19

4.7.3	Validation pure, découplée de la sortie HTTP	19
4.8	Composants d'accès aux données	20
4.9	Autres composants : le socle « framework »	21
4.10	Sécurité de l'application	22
4.10.1	XSS : échappement à la sortie	23
4.10.2	CSRF : jeton de synchronisation	23
4.10.3	En-têtes de sécurité et cookie de session	24
4.10.4	IDOR et contrôle d'accès	24
4.11	Plan de tests	25
4.12	Jeu d'essai de la fonctionnalité la plus représentative	25
4.13	Préparation et documentation du déploiement	26
4.14	Contribution à la mise en production (DevOps)	27
4.15	Veille de sécurité	27
5	Bilan, difficultés et perspectives	29
5.1	Satisfactions	29
5.2	Difficultés rencontrées et résolues	29
5.3	Perspectives	29
A	Annexes	31
A.1	Script de création de la base de données	31
A.2	Code d'un autre composant : le routeur	32
A.3	Jeux de tests complets	33
A.4	Maquettes et captures d'écran	34

Chapitre 1

Liste des compétences mises en œuvre

Ce dossier rend compte de la réalisation du projet SOLAGE, une application web de microblogging développée en formation, support de la mise en œuvre des **onze compétences professionnelles** du titre *Concepteur Développeur d'Applications* (RNCP 37873), réparties sur les trois certificats de compétences professionnelles (CCP).

Le tableau [1.1](#) constitue la *grille de lecture* du jury : chaque compétence y est reliée à la preuve concrète qui la démontre dans SOLAGE et au chapitre où elle est développée.

Compétences transversales. Elles sont évaluées à *travers* les compétences professionnelles :

- **Communiquer en français et en anglais** : dossier et présentation structurés, commentaires de code et vocabulaire technique en anglais (niveau B1) ;
- **Mettre en œuvre une démarche de résolution de problème** : voir le chapitre [5](#) (bugs diagnostiqués et résolus : `STDOUT CLI`, *headers already sent*, requête N+1) ;
- **Apprendre en continu** : veille de sécurité ayant conduit à la correction d'une faille IDOR (section [4.15](#)).

#	Compétence	CCP	Preuve principale dans Solage	Statut
C1	Installer et configurer son environnement de travail	1	Docker (FrankenPHP, Caddy, Traefik, PostgreSQL), Git, Composer, conteneurs dev = prod	OK
C2	Développer des interfaces utilisateur	1	Vues <code>modules/views/</code> , JS vanilla, AJAX, échappement XSS, CSP	OK
C3	Développer des composants métier	1	Contrôleurs <code>modules/controllers/</code> , autorisation, contrôle IDOR, CSRF	OK
C4	Contribuer à la gestion d'un projet informatique	1	Git, feuille de route, journal de décisions	Partiel
C5	Analyser les besoins et maquetter une application	2	Expression des besoins (ch. 2), maquettes & enchaînement	Partiel
C6	Définir l'architecture logicielle d'une application	2	MVC multicouche, Router, middlewares, schéma DICP	OK
C7	Concevoir et mettre en place une base de données relationnelle	2	<code>solage.pg.sql</code> , migrations idempotentes, <code>seed.sql</code>	OK
C8	Développer des composants d'accès aux données SQL et NoSQL	2	<code>modules/models/</code> , PDO requêtes préparées, anti-injection	OK
C9	Préparer et exécuter les plans de tests	3	Plan de tests + tests unitaires / sécurité (PHPUnit)	À développer
C10	Préparer et documenter le déploiement	3	<code>docker-compose.prod.yml</code> , Traefik HTTPS, procédure	Partiel
C11	Contribuer à la mise en production (DevOps)	3	Conteneurs, migrations, intégration continue (CI)	Partiel

Table 1.1 — Cartographie des onze compétences professionnelles et de leur preuve dans SOLAGE.

Chapitre 2

Expression des besoins

SOLAGE étant un projet réalisé *pendant la formation*, sans commanditaire externe, c’est moi qui formule l’expression des besoins définissant les objectifs et les limites du projet, comme le prévoit le référentiel de certification.

2.1 Contexte et problème adressé

SOLAGE est un **réseau social de microblogging** inspiré de X (anciennement Twitter) : un utilisateur publie de courts messages, éventuellement accompagnés d’une image, auxquels les autres peuvent répondre dans un fil imbriqué et réagir par un « j’aime ».

Le besoin pédagogique sous-jacent est de se confronter à *l’ensemble des briques d’une application web sécurisée multicouche* sur un périmètre fonctionnel maîtrisable : authentification, opérations de création / lecture / mise à jour / suppression (CRUD), relations récursives (réponses imbriquées), téléversement de fichiers, recherche et modération. Ce périmètre, volontairement resserré, permet de traiter **la sécurité à chaque couche** plutôt que d’empiler des fonctionnalités superficielles.

2.2 Objectifs du projet

Les objectifs sont formulés de manière *vérifiable* :

- permettre à un visiteur de **s’inscrire**, de **se connecter** et de se déconnecter ;
- permettre à un utilisateur authentifié de **publier** un message (avec image facultative), d’y **répondre** dans un fil imbriqué, de **l’aimer**, de **rechercher** des utilisateurs et des messages, et d’**éditer son profil** ;
- permettre à un **administrateur** de **modérer** (supprimer tout contenu, accéder à une page d’administration) ;
- garantir, de manière transverse, la **sécurité applicative** (protection contre les failles XSS, CSRF, injection SQL, IDOR) conformément aux recommandations de l’OWASP et de l’ANSSI, ainsi que la conformité au RGPD et la prise en compte du RGAA.

2.3 Périmètre et limites

Le **hors-périmètre** est assumé explicitement : il traduit un arbitrage de maturité, et non un oubli. Ne font pas partie du projet :

- la messagerie privée entre utilisateurs ;
- les notifications en temps réel (WebSocket) ;
- la fédération inter-instances et les API publiques ouvertes ;
- toute fonctionnalité de paiement ou de monétisation.

2.4 Acteurs et cas d’usage

Trois acteurs interagissent avec le système, selon une relation de *généralisation* : l’administrateur est un utilisateur particulier, lui-même un visiteur authentifié.

Acteur	Cas d’usage principaux
Visiteur	Consulter la page de connexion, s’inscrire, se connecter
Utilisateur	Consulter le fil, publier, répondre, aimer, rechercher, éditer son profil, supprimer son propre contenu
Administrateur	Tous les cas de l’utilisateur <i>plus</i> la modération (supprimer tout contenu, page d’administration)

Table 2.1 – Acteurs et cas d’usage.

Quelques *user stories* représentatives, qui serviront de base au diagramme de cas d’utilisation (section 4.5) et au plan de tests (chapitre 4.11) :

- *En tant que* visiteur, *je veux* créer un compte *afin de* publier des messages.
- *En tant qu’*utilisateur, *je veux* publier un message avec une image *afin de* partager du contenu.
- *En tant qu’*utilisateur, *je veux* répondre à un message *afin de* participer à une conversation.
- *En tant qu’*administrateur, *je veux* supprimer n’importe quel message *afin de* modérer la plateforme.

2.5 Contraintes réglementaires et qualité

RGPD. L’application traite des données à caractère personnel (adresse email, mot de passe, contenus publiés). Le mot de passe n’est jamais stocké en clair : il est haché (`bcrypt`). La collecte est minimale (principe de minimisation). Des mentions légales et une information sur la finalité du traitement doivent accompagner la mise en production.

RGAA (accessibilité). Les interfaces visent un socle d’accessibilité : attributs `alt` sur les images, contrastes suffisants, libellés de formulaire, navigation au clavier et attributs `aria-*` sur les éléments interactifs (par exemple le champ de saisie de message porte `role="textinput"` et `aria-label`).

Éco-conception. Plusieurs choix limitent l’empreinte : minification des assets en production, pagination du fil (LIMIT 20), et suppression d’une requête N+1 réduisant le nombre d’allers-retours vers la base (section 4.8).

Chapitre 3

Environnement technique

3.1 Vue d'ensemble de la pile technique

SOLAGE repose sur une pile volontairement légère, dont je maîtrise chaque brique.

Couche	Technologie	Rôle
Langage	PHP 8.3 (<code>declare(strict_types=1)</code>)	Logique applicative
SGBD	PostgreSQL 16 via PDO	Persistance relationnelle
Serveur applicatif	FrankenPHP (Caddy intégré)	Exécution de PHP, service du statique
Reverse proxy / edge	Traefik v3	Routage, terminaison TLS (Let's Encrypt)
Conteneurisation	Docker, <code>docker compose</code>	Reproductibilité dev = prod
Dépendances	Composer	<code>minify</code> , <code>phpdotenv</code> , <code>psr/log</code>
Front-end	HTML, CSS, JavaScript vanilla	Interfaces et appels AJAX (<code>fetch</code>)
Gestion de versions	Git	Historique et traçabilité

Table 3.1 – Pile technique de SOLAGE.

3.2 Justification des choix techniques

Chaque choix est un compromis explicite, formulé selon le schéma « X plutôt que Y parce que Z, en acceptant le coût W ».

MVC maison plutôt qu'un framework (Symfony, Laravel). Objectif pédagogique : *comprendre et posséder* le routeur, l'autoloader et le cycle requête / réponse, plutôt que de déléguer à de la « magie » non maîtrisée. Coût assumé : réécrire des briques qu'un framework fournirait. En contexte professionnel, un framework serait retenu.

PostgreSQL plutôt que MariaDB. Typage strict, séquences (`SERIAL`), robustesse et conformité SQL. Le projet a d'ailleurs été *migré* de MariaDB vers PostgreSQL, ce qui a imposé d'adapter le schéma (`AUTO_INCREMENT` → `SERIAL`, `datetime` → `TIMESTAMP`).

FrankenPHP plutôt qu'Apache ou Nginx + PHP-FPM. Un seul binaire embarquant Caddy, terminaison TLS automatique, image Docker simple.

Traefik plutôt que Nginx en reverse proxy. Configuration *par labels Docker*, certificats Let's Encrypt automatiques, découverte dynamique des services.

PDO plutôt qu'une extension native. Abstraction portable et, surtout, **requêtes préparées** qui neutralisent l'injection SQL (section 4.8).

3.3 Poste de travail et outils

L'environnement de développement réunit : un IDE, **Git** pour la gestion de versions (historique de *commits* conventionnels, une fonctionnalité par série de commits), **Composer** pour les dépendances (`composer.json` / `composer.lock`), et une gestion rigoureuse des **secrets**. Les identifiants de base de données ne figurent jamais dans le code : ils sont chargés depuis un fichier `.env` (ignoré par Git) au moyen de la bibliothèque `vlucas/phpdotenv`, un gabarit `.env.example` documentant les variables attendues.

```

1 $envPath = dirname(__DIR__);
2 if (file_exists($envPath . '/.env')) {
3     $dotenv = Dotenv\Dotenv::createImmutable($envPath);
4     $dotenv->load();
5 }
6 // ... lecture de DB_HOST, DB_NAME, DB_USER, DB_PASSWORD ...
7 $dsn = "pgsql:host={$host};port={$port};dbname={$dbname};options='--
      client_encoding=UTF8'";
8 $options = [
9     PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
10    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
11    PDO::ATTR_EMULATE_PREPARES  => false,    // vraies requêtes préparées côté
      serveur
12 ];
13 $this->connection = new PDO($dsn, $user, $password, $options);

```

Listing 3.1 — Chargement des secrets et connexion PDO — `includes/database.php`

La désactivation de `ATTR_EMULATE_PREPARES` force de *vraies* requêtes préparées côté serveur PostgreSQL, renforçant la protection contre l'injection SQL.

3.4 Conteneurisation (Docker)

L'application est décrite par deux piles `docker compose` : l'une pour le développement, l'autre pour la production.

Développement. Traefik en HTTP, FrankenPHP en *bind-mount* du code (rechargement à chaud), PostgreSQL exposé sur le port 5432 (pour les outils locaux), et un service `migrate` exécuté une seule fois avant le démarrage de l'application. Un *healthcheck* PostgreSQL garantit l'ordre de démarrage.

```

1 app:
2   build: { context: ., dockerfile: Dockerfile }
3   image: solage-app

```

```
4     volumes:
5         - ./:/app          # rechargement du code à chaud en dev
6         - /app/vendor      # vendor/ provient de l'image
7     depends_on:
8         postgres: { condition: service_healthy }
9         migrate:  { condition: service_completed_successfully }
10    labels:
11        - traefik.enable=true
12        - traefik.http.routers.solage.rule=Host('localhost')
13
14    postgres:
15        image: postgres:16-alpine
16        healthcheck:
17            test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
18            interval: 5s
```

Listing 3.2 – Extrait du `docker-compose.yml` (développement) : service applicatif et base

Production. Traefik en HTTPS (Let's Encrypt, redirection HTTP → HTTPS), en-tête HSTS, FrankenPHP servi depuis l'image (sans *bind-mount*), et PostgreSQL *non exposé* (accessible uniquement sur le réseau interne). Le détail figure en section 4.13 (préparation du déploiement).

Chapitre 4

Réalisations

Ce chapitre, cœur du dossier, présente les réalisations qui mettent en œuvre les compétences : l’architecture logicielle, la gestion de projet, la conception (maquettes, modèle de données, diagrammes UML), le développement (interfaces, composants métier, accès aux données), la sécurité, puis les tests et la veille.

4.1 Architecture logicielle multicouche

SOLAGE suit une architecture **MVC multicouche**, où chaque couche porte une responsabilité unique *et* un rôle de sécurité. La figure 4.1 en donne la vue d’ensemble.

4.1.1 Règles de séparation des couches

Le découpage est imposé par des règles strictes que je me suis fixées, vérifiables à la lecture du code :

- **aucun SQL hors de `modules/models/`** : un contrôleur appelle une méthode de modèle, il n’écrit jamais de requête ;
- **aucune sortie (`echo`) hors de `modules/views/` et de l’entrée `public/index.php`** ;
- **aucun accès à `$_POST` / `$_GET` / `$_SESSION` hors des contrôleurs et de `SessionManager`** ;
- **aucun appel direct à `Database::getConnection()` hors des modèles, de Migrations et du `bootstrap`**.

Le dossier `src/` regroupe le code « framework » (le routeur, les middlewares, le logger, les utilitaires, le gestionnaire de session, les migrations), distinct du code « métier » (`modules/`). Cette séparation entre infrastructure réutilisable et domaine applicatif est un marqueur d’architecture mûre.

4.1.2 Rôle de sécurité de chaque couche (DICP)

La stratégie de sécurité se lit couche par couche selon les quatre indicateurs de l’ANSSI : **D**isponibilité, **I**ntégrité, **C**onfidentialité, **P**reuve.

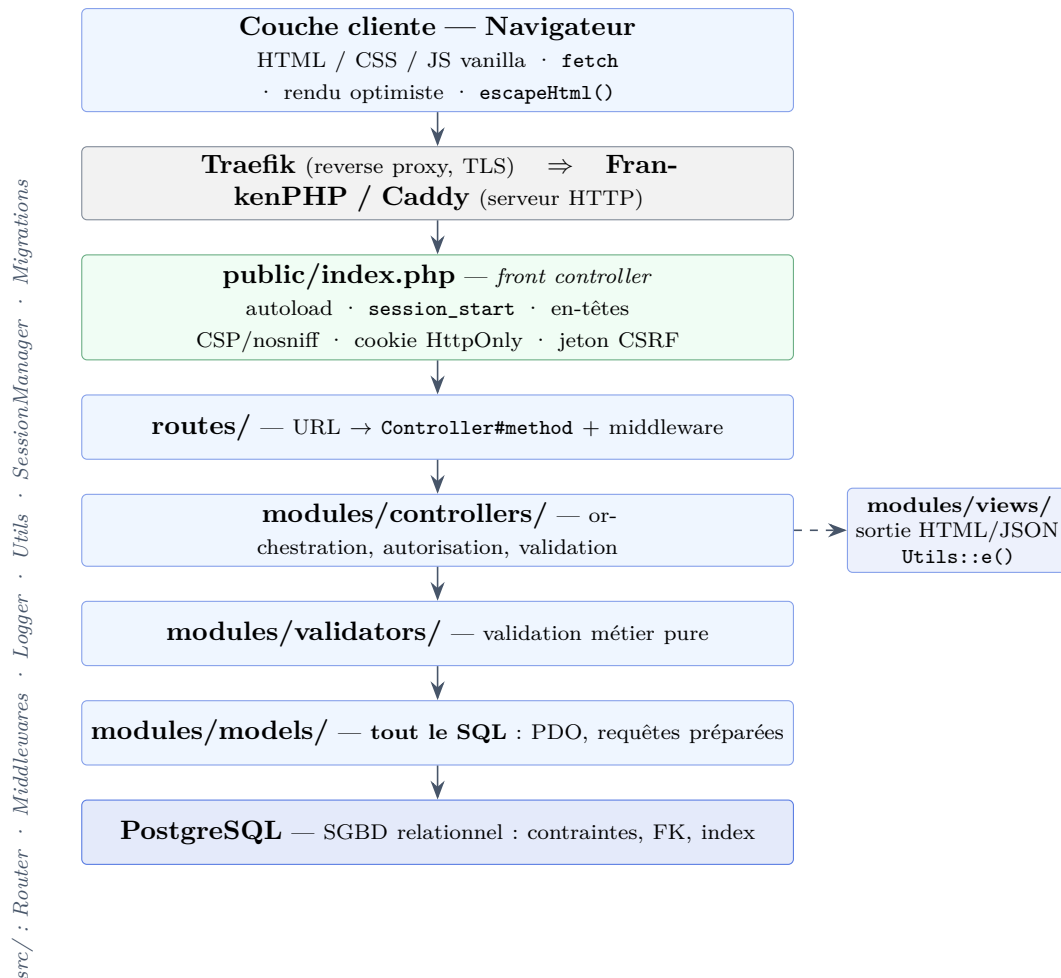


Figure 4.1 — Architecture multicouche de SOLAGE : du navigateur au SGBD.

4.1.3 Patrons mis en œuvre

L'architecture mobilise plusieurs patrons de conception : *Front Controller* (`public/index.php`, point d'entrée unique), *MVC* (séparation modèle / vue / contrôleur), *Middleware* (chaîne de traitement transverse pour l'authentification, l'autorisation et le CSRF) et *Injection de dépendance* (le `SessionManager` reçoit son `UserModel` par constructeur, ce qui le rend testable).

4.2 Gestion de projet

Le projet a été mené en autonomie selon une démarche **itérative légère** (proche de Kanban), structurée en phases successives : hygiène et sécurisation, conception, développement, tests, déploiement.

Planification et suivi. La feuille de route du projet (`ROADMAP_DETAILLEE.md`) décrit les phases, leur état d'avancement et les priorités au regard du risque d'échec par CCP. Git matérialise le suivi : chaque fonctionnalité correspond à une série de commits traçables, ce qui permet de rapprocher le réalisé du planifié.

Couche	Mécanisme de sécurité	Indicateur DICP
Edge (Traefik)	TLS / HTTPS, HSTS, isolation réseau	Intégrité, Confidentialité
Bootstrap (index.php)	En-têtes CSP / nosniff, cookie HttpOnly / Secure / SameSite	Intégrité, Confidentialité
Middlewares (src/)	Jeton CSRF, authentification, autorisation ; refus journalisé	Intégrité, Confidentialité, Preuve
Contrôleurs	Validation des entrées, contrôle d'accès par ressource (anti-IDOR)	Intégrité, Confidentialité
Modèles (PDO)	Requêtes préparées (anti-injection SQL), transactions	Intégrité
Vues	Échappement systématique <code>Utils::e()</code> (anti-XSS)	Intégrité
SGBD (PostgreSQL)	Contraintes, clés étrangères, index, comptes & droits	Disponibilité, Intégrité

Table 4.1 — Rôle de sécurité de chaque couche, selon les indicateurs DICP.

Comptes rendus et décisions. Un journal de décisions techniques (`Probleme-Solution.md`) consigne, pour chaque choix structurant, le contexte, le problème, les options envisagées, la solution retenue et sa justification. Ce journal tient lieu de comptes rendus de session et alimente la préparation de l'entretien technique.

À DÉVELOPPER — Gestion de projet — outil collaboratif et planning visuel

À intégrer pour compléter la compétence C4 :

- une capture d'un **outil collaboratif** (GitHub Projects / Trello) montrant le découpage en tâches et leur avancement ;
- un **planning visuel** (diagramme de Gantt ou feuille de route datée) confrontant le *planifié* au *réalisé* ;
- trois à quatre **comptes rendus** de sessions de travail (date, objectif, réalisé, reste à faire).

4.3 Maquettes et enchaînement des écrans

L'application comporte neuf écrans : connexion, inscription, fil d'actualité, détail d'un message, profil utilisateur, édition de profil, recherche, administration et page d'erreur 404. Leur enchaînement est formalisé par la figure 4.2.

La charte graphique retenue est minimaliste, conforme aux principes ergonomiques du référentiel (simplicité, minimalité des affichages). Le parcours nominal mène de la connexion au fil d'actualité, depuis lequel l'utilisateur accède au détail d'un message, aux profils, à la recherche et — s'il est administrateur — à la modération.

À DÉVELOPPER — Maquettes des interfaces (Figma)

Insérer ici les **maquettes** réalisées avec un outil de maquettage (Figma), au minimum pour

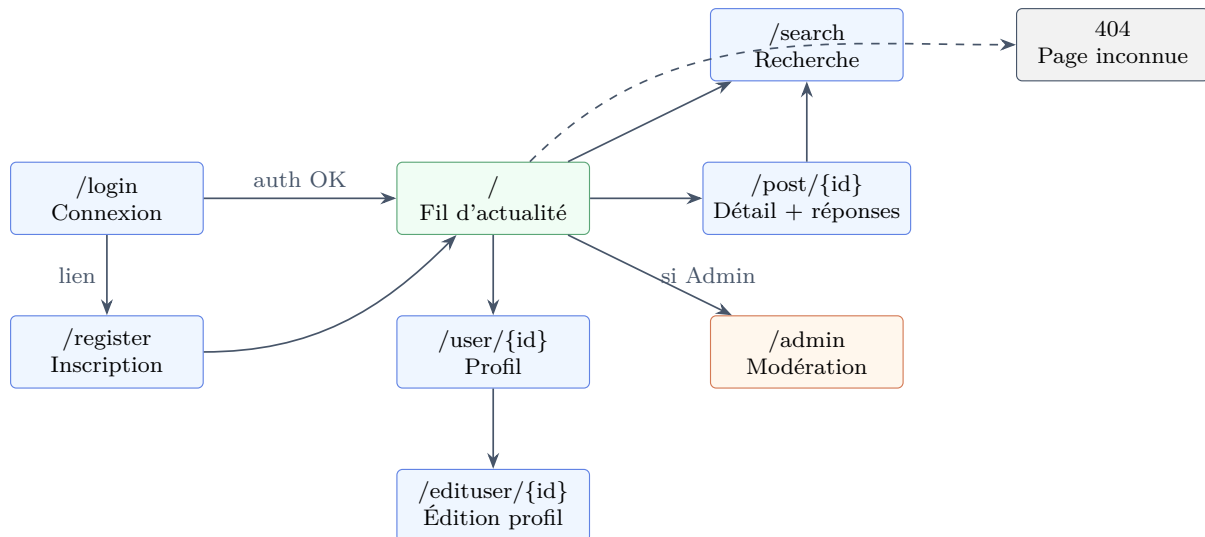


Figure 4.2 – Enchaînement des écrans (storyboard). En vert l'écran protégé par authentification, en orange l'écran réservé à l'administrateur.

les écrans *connexion*, *fil d'actualité* et *détail d'un message*. Pour chaque maquette : une capture, et un court paragraphe justifiant les choix d'ergonomie et de charte graphique (placement des actions, hiérarchie visuelle, accessibilité). Les maquettes des autres écrans iront en annexe. *Critère C5* : « les maquettes sont réalisées conformément au cahier des charges » et « l'enchaînement est formalisé par un schéma » (déjà fourni ci-dessus).

4.4 Conception de la base de données

4.4.1 Modèle conceptuel des données (MCD)

Le modèle conceptuel (figure 4.3) recense trois entités principales : UTILISATEUR, ROLE et POST. Les réponses sont modélisées par une association *réflexive* « répond à » sur POST : une réponse *est* un message qui en référence un autre. Les « j'aime » et les favoris sont des associations entre UTILISATEUR et POST, l'association « aime » portant l'attribut `created_at`.

Note. Le schéma physique conserve une table `responses` héritée d'une première version, alors que le mécanisme de réponse effectivement utilisé par l'application repose sur les colonnes `reply_to` / `reply_to_parent` de la table `posts`. Cette redondance est identifiée comme une dette à résorber : c'est un point que je signale plutôt que de le masquer.

4.4.2 Modèle logique / physique (MLD, MPD)

Le passage au modèle logique transforme les associations en clés étrangères. La règle de nommage notable : le mot `user` étant *réservé* en PostgreSQL, les colonnes de clé étrangère se nomment `user_id`. La figure 4.4 présente le schéma relationnel et ses clés étrangères.

Le modèle physique est concrétisé par le script de création `solage.pg.sql`, chargé une seule fois à l'initialisation du conteneur PostgreSQL. L'extrait suivant en montre les tables principales ;

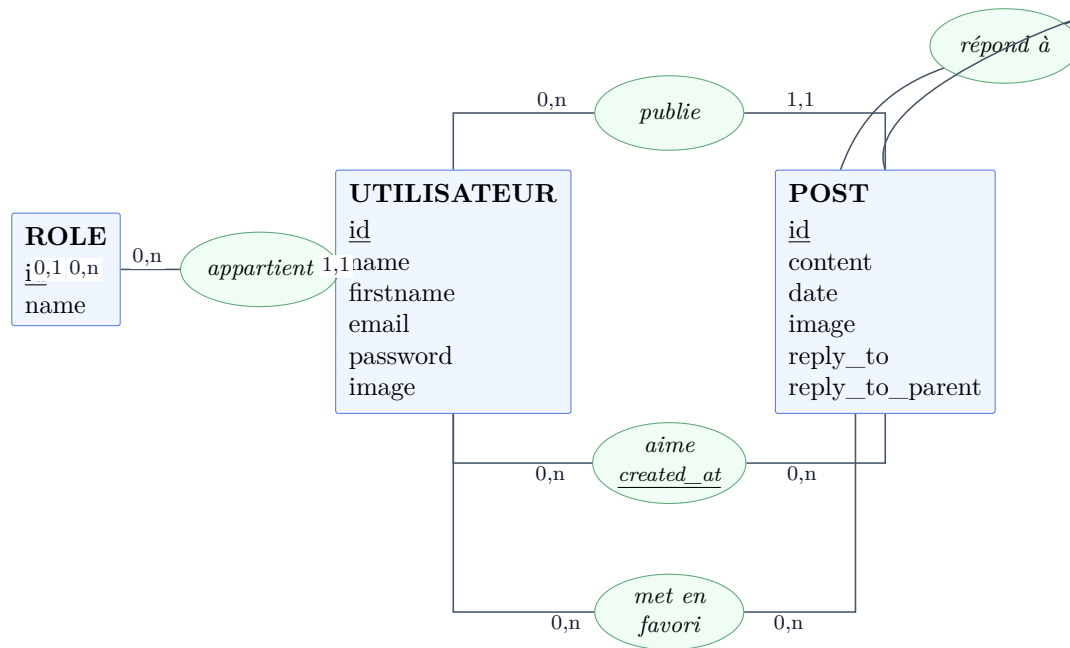


Figure 4.3 – Modèle conceptuel des données (formalisme Merise).

le script complet figure en annexe A.1.

```

1 CREATE TABLE roles (
2     id SERIAL PRIMARY KEY,
3     name VARCHAR(255) NOT NULL
4 );
5
6 CREATE TABLE users (
7     id SERIAL PRIMARY KEY,
8     name VARCHAR(255) NOT NULL,
9     email VARCHAR(255) NOT NULL UNIQUE,
10    password VARCHAR(255) NOT NULL,
11    role INT NULL REFERENCES roles(id) ON DELETE SET NULL,
12    image VARCHAR(255) NULL
13 );
14 CREATE INDEX users_role_idx ON users(role);
15
16 CREATE TABLE posts (
17     id SERIAL PRIMARY KEY,
18     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
19     date TIMESTAMP NOT NULL,
20     content TEXT NULL,
21     reply_to INT NULL,
22     reply_to_parent INT NULL,
23     image TEXT NULL
24 );
25 CREATE INDEX posts_user_idx ON posts(user_id);
26
27 CREATE TABLE likes (
28     id SERIAL PRIMARY KEY,
29     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
  
```

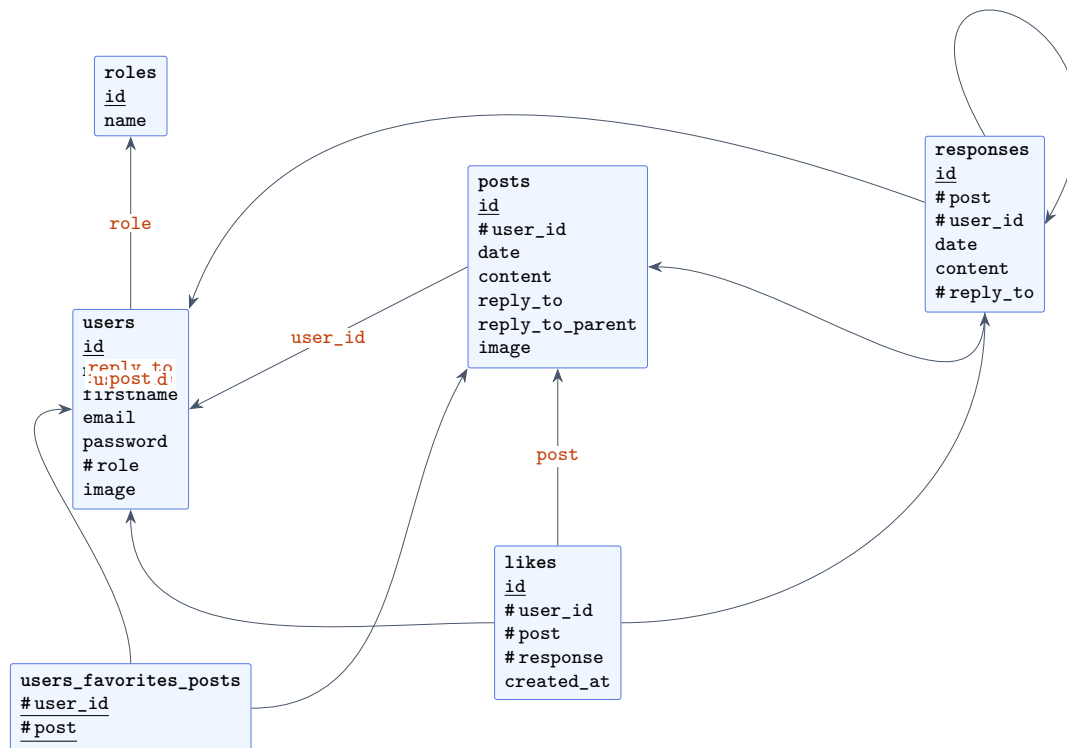



Figure 4.4 – Modèle logique des données : tables physiques et clés étrangères (#).

```

30     post      INT NULL REFERENCES posts(id) ON DELETE CASCADE,
31     response  INT NULL REFERENCES responses(id) ON DELETE CASCADE,
32     created_at TIMESTAMP NOT NULL
33 );

```

Listing 4.1 – Extrait du script de création — solage.pg.sql

L'intégrité est garantie par les contraintes : NOT NULL, UNIQUE(email), clés étrangères avec ON DELETE CASCADE (suppression en cascade des messages d'un utilisateur supprimé) ou SET NULL. Des **index** accélèrent les accès fréquents (clés étrangères, fil d'actualité).

4.4.3 Évolution du schéma : migrations idempotentes

Les évolutions additives ne sont pas appliquées à la main : elles passent par un système de **migrations idempotentes** (src/Migrations.php), qui interroge `information_schema` avant d'agir et peut donc être rejoué sans risque.

```

1  protected function columnExists(string $table, string $column): bool {
2      $sql = "SELECT 1 FROM information_schema.columns
3              WHERE table_schema = current_schema()
4              AND table_name = :table AND column_name = :column LIMIT 1";
5      $stmt = $this->db->prepare($sql);
6      $stmt->execute([':table' => $table, ':column' => $column]);
7      return $stmt->fetchColumn() !== false;
8  }

```

```

9
10 public function migrate(): void {
11     $this->addColumnIfNotExists('posts', 'reply_to', 'INT', 'NULL');
12     $this->addColumnIfNotExists('posts', 'reply_to_parent', 'INT', 'NULL');
13     $this->addColumnIfNotExists('posts', 'image', 'TEXT', 'NULL');
14 }

```

Listing 4.2 – Migration idempotente — src/Migrations.php

Jeu d'essai et restauration. Un jeu d'essai reproductible (`seed.sql`) peuple une base de test (utilisateurs, messages, « j'aime », réponses).

À DÉVELOPPER — Procédure de sauvegarde / restauration

Documenter et illustrer la procédure de sauvegarde et de restauration de la base (`pg_dump` / `pg_restore`), exigée par le critère C7 (« la base de test peut être restaurée en cas d'incident ») : commande de *dump*, emplacement de stockage, commande de restauration, et test de restauration sur une base vierge.

4.5 Diagrammes UML

4.5.1 Diagramme de cas d'utilisation

La figure 4.5 formalise le comportement attendu : les trois acteurs et leurs cas d'usage, avec la généralisation Administrateur → Utilisateur → Visiteur.

4.5.2 Diagramme de séquence : « Publier un message »

Le cas le plus significatif est la publication d'un message : il traverse toutes les couches et mobilise les protections de sécurité. La figure 4.6 en détaille le déroulé, de l'appel `fetch` du navigateur jusqu'à l'insertion en base et au rendu optimiste.

4.5.3 Diagramme de classes (couche modèle)

La figure 4.7 présente un extrait de la couche modèle et le gestionnaire de session, avec leurs principales méthodes et dépendances.

4.6 Composants d'interface utilisateur

Les interfaces sont rendues par la couche `modules/views/`. Une vue ne contient aucune logique d'accès aux données : elle reçoit des données déjà préparées par le contrôleur et se contente de produire du HTML, en **échappant systématiquement** tout contenu issu de l'utilisateur via `Utils::e()`.

```

1 <div class="createPost">
2     <div class="postAvatar"><?= Utils::e($userImage) ?></div>
3     <div class="postInsideContainer">

```

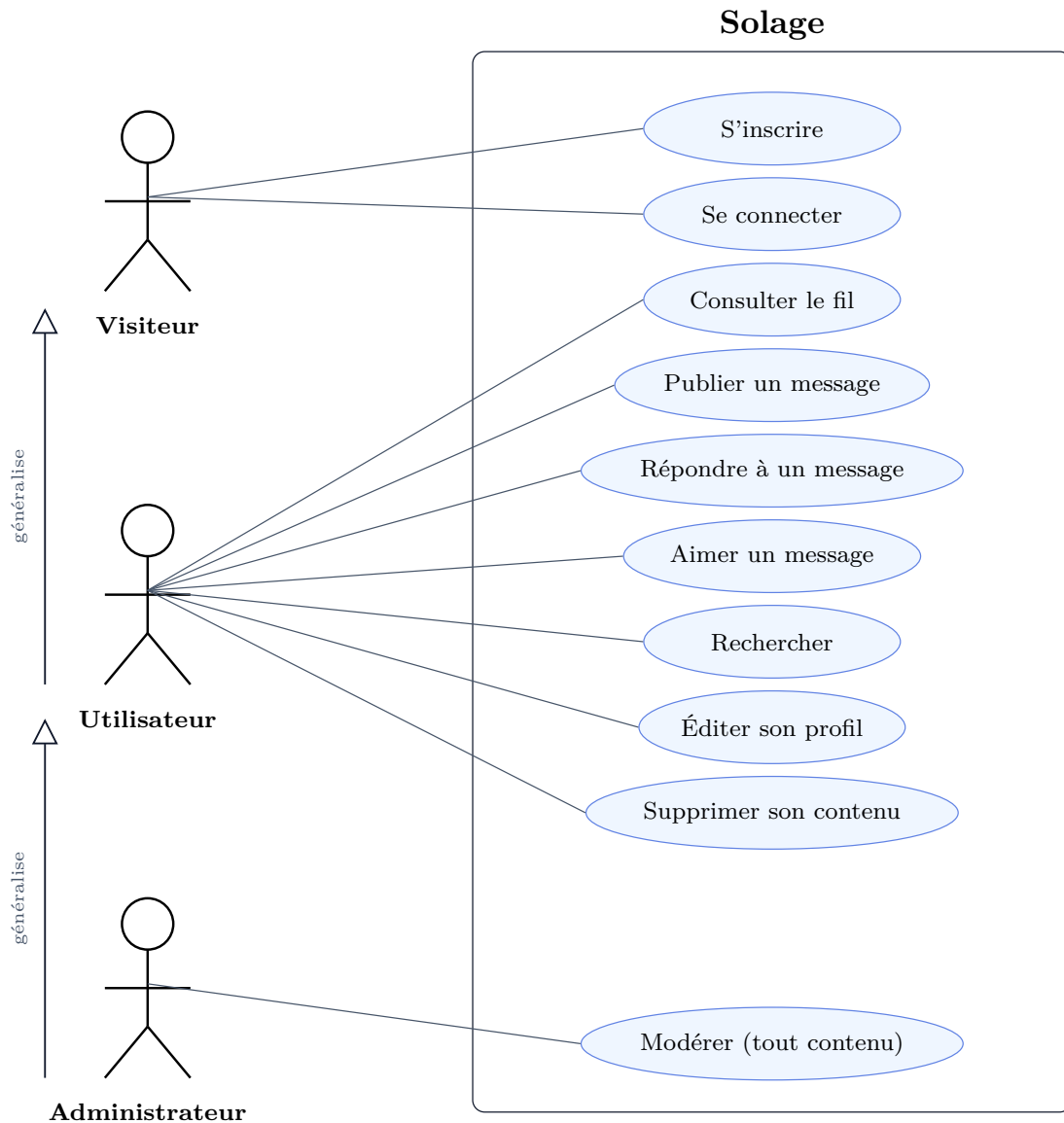


Figure 4.5 – Diagramme de cas d'utilisation de SOLAGE.

```

4      <div class="postNameDate"><?= Utils::e($username) ?></div>
5      <span id="postContent" aria-label="Contenu du post"
6          role="textbox" contenteditable="true"></span>
7      <input id="file-input" accept="image/*" type="file" class="hidden"
8          />
9      <button id="postCreateButton" class="postCreateButton">Publier</
10     button>
11 </div>
12 </div>

```

Listing 4.3 – Vue du formulaire de publication (extrait) — modules/views/CreatePostView.php

Asynchronisme (AJAX). Les actions (publier, aimer, supprimer, se connecter) sont réalisées sans rechargement de page, par `fetch`. Après création d'un message, le DOM est mis à jour de façon *optimiste* : le message est injecté immédiatement, sans attendre un rechargement.

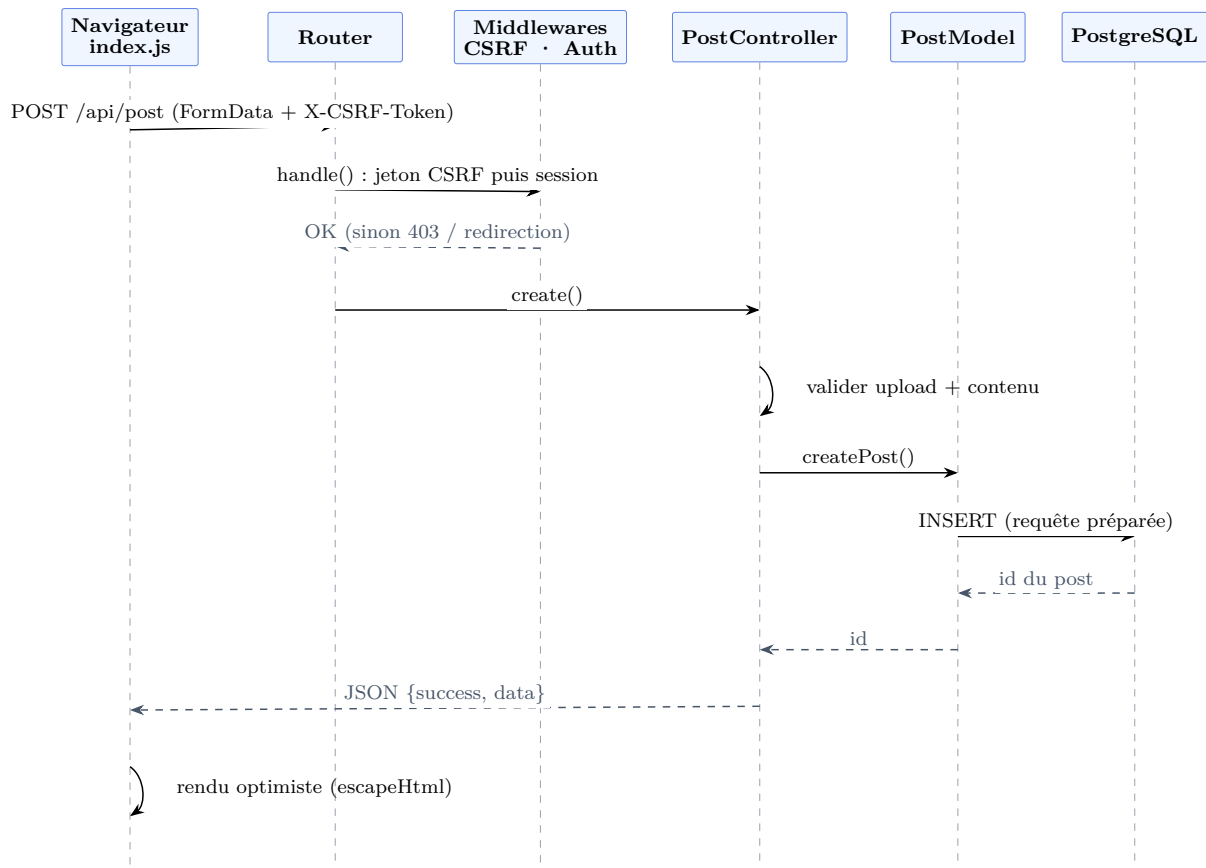


Figure 4.6 – Diagramme de séquence : publication d'un message (avec passage par les middlewares CSRF et authentification).

Comme cette injection passe par `innerHTML`, un échappement *côté client* (`escapeHtml`), miroir de `Utils::e()`, est indispensable (voir la section 4.10).

```

1  const content = document.getElementById("postContent").innerText.trim();
2  if (!content) { alert("Le contenu du post ne peut pas être vide !"); return; }
3
4  const formData = new FormData();
5  formData.append('data', JSON.stringify({ content, replyTo, replyToParent }));
6  if (selectedImage) formData.append('image', selectedImage);
7
8  fetch("/api/post", { method: "POST", body: formData }) // le jeton CSRF est
   injecté
9  .then(response => response.json()) // automatiquement (
   cf. ch. securite)
10 .then(data => { if (data.success) { /* rendu optimiste via escapeHtml */ }
   });
  
```

Listing 4.4 – Création d'un message par fetch — `public/scripts/index.js`

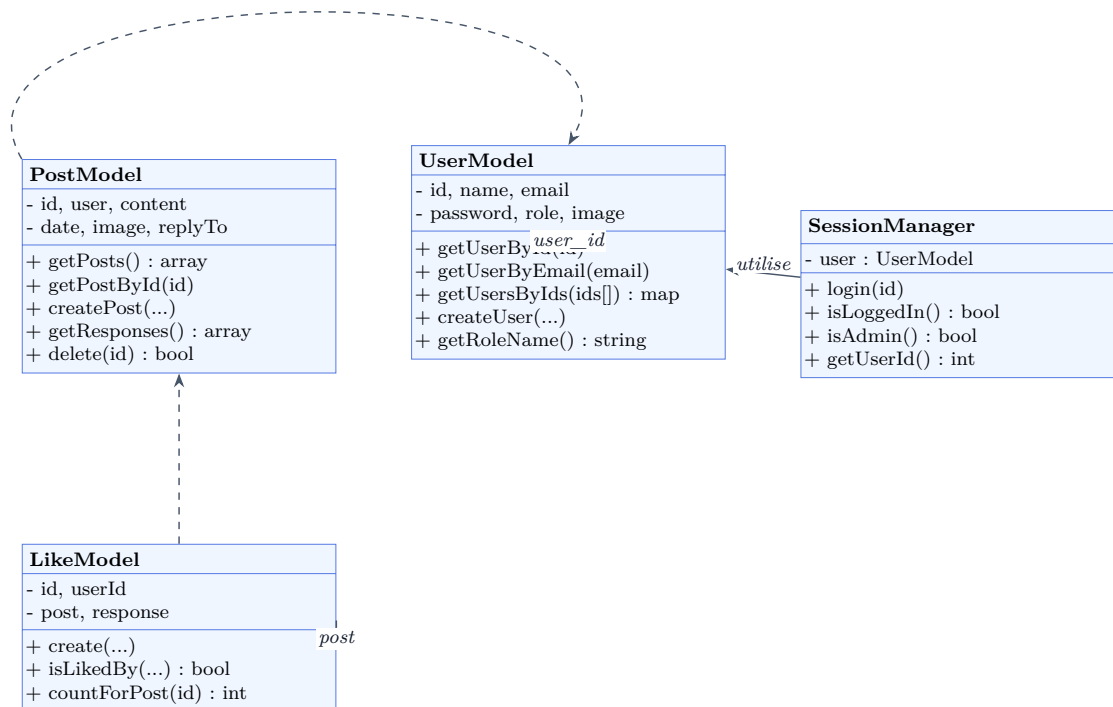


Figure 4.7 — Diagramme de classes (extrait) : modèles et gestion de session.

À DÉVELOPPER — Captures d'écran des interfaces

Insérer ici une à deux **captures d'écran** des interfaces réelles (le fil d'actualité et le formulaire de publication), mises en regard du code de la vue correspondante. Le référentiel (C2) attend explicitement « les captures d'écran d'interfaces utilisateur et le code correspondant ». Les autres captures iront en annexe.

4.7 Composants métier

Les composants métier vivent dans `modules/controllers/`. Ils orchestrent la requête, valident les entrées et — point essentiel — vérifient l'**autorisation sur la ressource**, et non la seule authentification.

4.7.1 Contrôle d'accès : la parade à l'IDOR

La suppression d'un message illustre le contrôle d'accès. Avant toute suppression, le contrôleur vérifie que l'utilisateur courant est *propriétaire* de la ressource ou administrateur ; sinon, il répond 403 et journalise la tentative.

```

1 $session = new SessionManager(new UserModel());
2 $userId  = $session->getUserId();
3 $post    = PostModel::getPostById($postId);
4
5 if ($post->getUserId() !== $userId && !$session->isAdmin()) {
6     Logger::get()->warning('post.delete.forbidden', [

```

```

7         'current_user_id' => $userId,
8         'post_owner_id'   => $post->getUserId(),
9         'post_id'         => $postId,
10    ]);
11    http_response_code(403);
12    Utils::sendResponse(false, "Vous n'avez pas la permission de supprimer
    ce post.");
13    return;
14 }
15 $result = PostModel::delete($postId);

```

Listing 4.5 – Contrôle d'autorisation anti-IDOR — modules/controllers/PostController.php

4.7.2 Authentification vs autorisation

La distinction est portée par le `SessionManager` : `isLoggedIn()` répond à « qui es-tu ? » (authentification), `isAdmin()` répond à « qu'as-tu le droit de faire ? » (autorisation, résolue par le *libellé* du rôle et non par un identifiant numérique magique).

```

1 public function isLoggedIn(): bool {
2     return isset($_SESSION['user_id']);
3 }
4 public function isAdmin(): bool {
5     return $this->user !== null && $this->user->getRoleName() === 'Admin';
6 }

```

Listing 4.6 – Authentification et autorisation — src/SessionManager.php

4.7.3 Validation pure, découplée de la sortie HTTP

Le validateur (modules/validators/UserValidator.php) *décide* sans produire de sortie : il retourne un tableau structuré. C'est le contrôleur qui décide ensuite quoi en faire. Ce découplage rend le validateur **testable** (il n'émet ni `echo` ni en-tête).

```

1 public static function login($email, $password): array {
2     if (empty($email) || empty($password)) {
3         return ['ok' => false, 'type' => 'error', 'message' => "Merci de
    renseigner vos informations"];
4     }
5     $user = (new UserModel())->getUserByEmail($email);
6     if (!$user) {
7         return ['ok' => false, 'type' => 'error', 'message' => "L'email n'
    existe pas"];
8     }
9     if (!password_verify($password, $user->getPassword())) {
10        return ['ok' => false, 'type' => 'error', 'message' => "Le mot de
    passe ne correspond pas"];
11    }
12    return ['ok' => true, 'type' => 'success', 'message' => "Vous vous êtes
    bien connecté !"];

```

```
13 }
```

Listing 4.7 — Validation pure (retourne un tableau) — `UserValidator.php`

4.8 Composants d'accès aux données

Toute la persistance vit dans `modules/models/`. Les requêtes sont **préparées** : les valeurs utilisateur partent en paramètres liés, jamais concaténées dans la requête — l'injection SQL est structurellement impossible.

```
1 public function createPost(?int $replyTo = null, ?int $replyToParent = null)
2 {
3     try {
4         $statement = $this->db->prepare(
5             'INSERT INTO posts (user_id, content, date, reply_to, image,
6               reply_to_parent)
7             VALUES (:user_id, :content, :date, :reply_to, :image, :
8               reply_to_parent)');
9         $statement->bindValue(':user_id', $this->user);
10        $statement->bindValue(':content', $this->content);
11        $statement->bindValue(':date', $this->date);
12        $statement->bindValue(':reply_to', $replyTo, PDO::PARAM_INT);
13        $statement->bindValue(':image', $this->image);
14        $statement->bindValue(':reply_to_parent', $replyToParent, PDO::
15            PARAM_INT);
16        $statement->execute();
17        return $this->db->lastInsertId('posts_id_seq');
18    } catch (PDOException $e) {
19        Logger::get()->error('post.create.failed', ['user_id' => $this->user
20            , 'exception' => $e]);
21        return false;
22    }
23 }
```

Listing 4.8 — Insertion par requête préparée — `modules/models/PostModel.php`

Optimisation : suppression d'une requête N+1. Afficher l'auteur de chaque message provoquait, à l'origine, une requête par message (21 requêtes pour 20 messages). La méthode `getUsersByIds` précharge tous les auteurs en *une* requête IN, ramenant le coût à deux requêtes. Le cas IN () vide — une erreur SQL en PostgreSQL — est court-circuité.

```
1 public static function getUsersByIds(array $ids): array {
2     if (empty($ids)) { // IN () est une erreur SQL en
3         PostgreSQL
4         return [];
5     }
6     $placeholders = implode(',', array_fill(0, count($ids), '?'));
7     $stmt = Database::getConnection()->prepare(
```

```

7      "SELECT id, name, email, password, role, image FROM users WHERE id
      IN ($placeholders)");
8      $stmt->execute(array_values($ids));    // valeurs liées : pas d'
      injection possible
9      $users = [];
10     while ($row = $stmt->fetch(PDO::FETCH_OBJ)) {
11         $users[(int)$row->id] = new UserModel($row);
12     }
13     return $users;
14 }

```

Listing 4.9 — Préchargement des auteurs en une requête — modules/models/UserModel.php

4.9 Autres composants : le socle « framework »

Le dossier `src/` contient le socle que j'ai écrit : routeur, middlewares, logger, utilitaires. C'est la preuve de maîtrise architecturale — chaque ligne est défendable.

Routeur. Le routeur associe une URL (avec paramètres en expression régulière) à un `Controller#method`, exécute le middleware de route, puis **arme le contrôle CSRF sur toute requête POST** — sécurité par défaut plutôt que sur option.

```

1  if ($route['method'] === $requestMethod && preg_match($pathRegex,
    $requestUri, $matches)) {
2      if ($route['middleware']) {
3          (new $route['middleware'])->handle();    // Auth ou Admin, selon
          la route
4      }
5      if ($route['method'] === 'POST') {
6          (new CsrMiddleware())->handle();    // CSRF armé sur
          TOUTES les mutations
7      }
8      // ... instantiation du contrôleur et appel de la méthode ...
9  }

```

Listing 4.10 — Dispatch et armement du CSRF sur les POST — src/Router.php

Middlewares : authentification vs autorisation. Deux classes distinctes séparent explicitement « être connecté » et « être administrateur ». Le refus d'accès administrateur est journalisé (indicateur de *preuve* du DICP).

```

1  public function handle() {
2      $session = new SessionManager(new UserModel());
3      if (!$session->isLoggedIn()) { header('Location: /login'); exit; }
4      if (!$session->isAdmin()) {
5          Logger::get()->warning('admin.access.denied', [
6              'user_id' => $session->getUserId(), 'uri' => $_SERVER['
              REQUEST_URI'] ?? '');
7          http_response_code(403);

```



```

8         echo "403 -- accès réservé aux administrateurs."; exit;
9     }
10 }

```

Listing 4.11 – Middleware d'autorisation administrateur — src/AdminMiddleware.php

Journalisation PSR-3. Le logger (src/Logger.php) implémente le standard PSR-3 et écrit des enregistrements *JSON-line* sur la sortie standard (et la sortie d'erreur pour les niveaux **warning** et au-delà), conformément aux conventions Docker : c'est l'orchestrateur qui collecte les journaux, l'application ne gère pas de fichiers de log.

```

1 public function log($level, string|\Stringable $message, array $context =
    []): void {
2     $record = ['ts' => date('c'), 'level' => (string) $level,
3               'msg' => $this->interpolate((string) $message, $context)];
4     if ($context !== []) { $record['context'] = $this->serializeContext(
5         $context); }
6     // STDOUT/STDERR n'existent qu'en CLI ; les flux php://* marchent
7     // partout (CLI, HTTP, FrankenPHP)
8     $target = in_array($level, self::STDERR_LEVELS, true) ? 'php://stderr' :
9         'php://stdout';
10    file_put_contents($target, json_encode($record, JSON_UNESCAPED_UNICODE)
11        . "\n", FILE_APPEND);
12 }

```

Listing 4.12 – Logger PSR-3, sortie JSON sur php://stdout — src/Logger.php

4.10 Sécurité de l'application

La sécurité est traitée **faillie par faille**, en référence à l'OWASP Top 10 et aux recommandations de l'ANSSI. Le tableau 4.2 résume les menaces couvertes et leur parade dans SOLAGE.

Menace (OWASP)	Parade dans Solage	Fichier(s)
XSS stocké (A03)	Échappement à la sortie : <code>Utils::e()</code> (serveur) + <code>escapeHtml()</code> (client)	<code>vues</code> , <code>index.js</code>
CSRF	Jeton de synchronisation armé sur tout POST, comparaison à temps constant	<code>CsrfMiddleware</code> , <code>CsrfHelper</code>
Injection SQL (A03)	Requêtes préparées PDO partout	<code>modules/models/</code>
Contrôle d'accès / IDOR (A01)	Vérification « propriétaire ou administrateur » + 403 + journal	<code>PostController</code> , <code>UserController</code>
Mots de passe	<code>password_hash</code> / <code>password_verify</code> (bcrypt)	<code>UserModel</code>
En-têtes	CSP, <code>nosniff</code> , <code>Referrer-Policy</code> , HSTS (prod)	<code>index.php</code> , prod
Cookie de session	<code>HttpOnly</code> + <code>SameSite=Lax</code> + <code>Secure</code> (prod)	<code>index.php</code>

Table 4.2 – Couverture des principales failles web.

4.10.1 XSS : échappement à la sortie

Le choix d'architecture est d'échapper **à la sortie**, pas à l'entrée : on conserve la donnée originale (un utilisateur peut légitimement écrire « < ») et on neutralise au moment de l'affichage. Côté serveur, `Utils::e()` encapsule `htmlspecialchars`; côté client, `escapeHtml()` en est le miroir, requis car le JavaScript reconstruit du DOM.

```
1 public static function e(?string $value): string {
2     return htmlspecialchars($value ?? '', ENT_QUOTES | ENT_HTML5, 'UTF-8');
3 }
```

Listing 4.13 – Échappement côté serveur — `src/Utils.php`

4.10.2 CSRF : jeton de synchronisation

La protection CSRF suit le patron *Synchronizer Token* : un secret par session, généré par un générateur cryptographique (`random_bytes`), exposé dans une balise `<meta>` et renvoyé par le client dans l'en-tête `X-CSRF-Token`. La vérification compare en **temps constant** (`hash_equals`), ce qui protège des attaques temporelles.

```
1 public static function getToken(): string {
2     if (empty($_SESSION['csrf_token'])) {
3         $_SESSION['csrf_token'] = bin2hex(random_bytes(32)); // CSPRNG,
3             pas uniqid()/rand()
4     }
5     return $_SESSION['csrf_token'];
6 }
7 public static function verifyToken(?string $token): bool {
8     return !empty($_SESSION['csrf_token'])
9         && hash_equals($_SESSION['csrf_token'], (string) $token); // temps
9             constant
10 }
```

Listing 4.14 – Génération et vérification du jeton CSRF — `src/CsrfHelper.php`

Le jeton est armé par le routeur sur *toutes* les requêtes POST (section 4.9), et injecté automatiquement dans chaque `fetch` par une enveloppe unique côté client :

```
1 const token = document.querySelector('meta[name="csrf-token"]')?.content;
2 const nativeFetch = window.fetch.bind(window);
3 window.fetch = function (resource, options = {}) {
4     const method = (options.method || 'GET').toUpperCase();
5     const safe = method === 'GET' || method === 'HEAD' || method === '
5         OPTIONS';
6     if (token && !safe) {
7         const headers = new Headers(options.headers || {});
8         headers.set('X-CSRF-Token', token);
9         options = { ...options, headers };
10    }
11    return nativeFetch(resource, options);
12 };
```

Listing 4.15 — Injection automatique du jeton dans tout fetch — public/scripts/index.js

Note. La protection est de la *défense en profondeur* : le cookie de session est déjà en SameSite=Lax (protection *navigateur*), et le jeton de synchronisation ajoute une protection *applicative*, indépendante du navigateur. On garde les deux.

4.10.3 En-têtes de sécurité et cookie de session

Le point d'entrée public/index.php pose les en-têtes applicatifs et durcit le cookie de session *avant* tout démarrage de session (les paramètres du cookie se figent au démarrage).

```

1 session_set_cookie_params([
2     'httponly' => true,                // inaccessible au JS (anti-vol
        de session par XSS)
3     'samesite' => 'Lax',                // anti-CSRF de base
4     'secure'   => APP_ENV === 'production', // HTTPS uniquement en
        production
5 ]);
6 header("X-Content-Type-Options: nosniff"); // anti
    MIME-sniffing
7 header("Referrer-Policy: strict-origin-when-cross-origin"); // pas de
    fuite d'URL
8 header("Content-Security-Policy: default-src 'self'; img-src 'self' data:;")
    ;
9 session_start();

```

Listing 4.16 — En-têtes et cookie de session — public/index.php

L'en-tête **HSTS** (Strict-Transport-Security) est posé à la couche TLS, par Traefik en production, car c'est lui qui termine le HTTPS — et non PHP, qui ne voit que du HTTP interne.

4.10.4 IDOR et contrôle d'accès

La parade à l'IDOR (*Insecure Direct Object Reference*, OWASP A01) est détaillée en section 4.7 : un utilisateur ne peut modifier ou supprimer que ses propres ressources. Cette faille — et le cheminement qui a conduit à la corriger — est documentée dans la veille de sécurité (section 4.15).

À DÉVELOPPER — Renforcements de sécurité planifiés

Honnêteté assumée : trois renforcements restent à mettre en œuvre et seront présentés comme axes d'amélioration.

- **Validation serveur systématique** : durcir UserValidator (format de l'email, robustesse du mot de passe : longueur, majuscule, chiffre) — aujourd'hui partielle.
- **Anti-fixation de session** : appeler session_regenerate_id(true) après la connexion.
- **Validation d'upload par type MIME réel** (finfo + getimagesize) en complément

de l'extension, et limitation de la taille.

4.11 Plan de tests

La stratégie de tests couvre quatre niveaux : tests **unitaires** (composants isolés), tests d'**intégration** (parcours traversant plusieurs couches), tests de **sécurité** (rejouant les failles corrigées) et tests de **non-régression**. Le tableau 4.3 présente le plan, dérivé des fonctionnalités et des *user stories* de la section 2.4.

Fonctionnalité	Type de test	Objet du test
Connexion	Unitaire	<code>UserValidator::login</code> (champs vides, email inconnu, mauvais mot de passe)
Échappement	Unitaire	<code>Utils::e()</code> neutralise <code><script></code>
Accès données	Unitaire	<code>PostModel</code> : création, lecture, suppression
Routage	Unitaire	<code>Router</code> : correspondance d'URL et paramètres
Publier un message	Intégration	Parcours complet : <code>fetch</code> → contrôleur → modèle → base
Injection SQL	Sécurité	Recherche avec charge utile : la requête préparée neutralise
XSS	Sécurité	Message <code><script></code> : rendu échappé, non exécuté
CSRF	Sécurité	POST sans jeton → 403
IDOR	Sécurité	Suppression d'un message d'autrui → 403

Table 4.3 – Plan de tests de SOLAGE.

À DÉVELOPPER — Implémentation et exécution des tests (PHPUnit)

Cette section doit être complétée par l'implémentation réelle — c'est la priorité du projet, la compétence C9 étant **obligatoire**. À produire :

1. installer **PHPUnit** (dépendance de développement Composer) ;
2. créer un **environnement de test** : conteneur PostgreSQL dédié (ou SQLite en mémoire via un adaptateur) ;
3. écrire les **tests unitaires** listés ci-dessus et capturer la sortie verte ;
4. écrire les **quatre tests de sécurité** (injection SQL, XSS, CSRF, IDOR) ;
5. insérer une **capture du rapport d'exécution** (nombre de tests, assertions, verts / rouges) ;
6. renseigner la colonne « résultat obtenu » du jeu d'essai (section suivante).

Code de test, sorties complètes et jeux de données iront en annexe A.3.

4.12 Jeu d'essai de la fonctionnalité la plus représentative

La fonctionnalité retenue est « **Publier un message** » : elle traverse toutes les couches (interface, contrôleur, validation, téléversement, modèle, SQL, réponse, rendu) et concentre les protections de sécurité. Le tableau 4.4 en présente le jeu d'essai.

Cas	Donnée d'entrée	Résultat attendu	Obtenu
Nominal	Contenu « Bonjour »	Message créé, id retourné, HTTP 200	(à exécuter)
Contenu vide	""	Refus « contenu vide », pas d'insertion	(à exécuter)
Image trop lourde	Fichier > limite	Refus « image trop lourde »	(à exécuter)
Extension interdite	Fichier .php	Refus « type de fichier non autorisé »	(à exécuter)
Charge XSS	<script>...	Stocké brut, rendu échappé	(à exécuter)
Non authentifié	Aucune session	Redirection / 403 (AuthMiddleware)	(à exécuter)
Sans jeton CSRF	POST sans jeton	403 (CsrfMiddleware)	(à exécuter)

Table 4.4 – Jeu d'essai de la fonctionnalité « Publier un message ».

À DÉVELOPPER — Colonne « résultat obtenu » et analyse des écarts

Après exécution, renseigner la colonne « Obtenu » pour chaque cas et ajouter, sous le tableau, l'**analyse des écarts** éventuels (cas où l'obtenu diffère de l'attendu, cause, correctif). Le référentiel exige cette analyse, même en l'absence d'écart (« analyse des écarts éventuels »).

4.13 Préparation et documentation du déploiement

La mise en production s'appuie sur la pile `docker-compose.prod.yml` : Traefik termine le HTTPS (Let's Encrypt), redirige HTTP vers HTTPS et pose l'en-tête HSTS ; le service `migrate` applique les migrations *avant* le démarrage de l'application ; PostgreSQL n'est pas exposé hors du réseau interne.

```

1 labels:
2   - traefik.http.routers.solage.entrypoints=websecure
3   - traefik.http.routers.solage.tls.certresolver=le
4   - traefik.http.middlewares.solage-hsts.headers.stsSeconds=31536000
5   - traefik.http.middlewares.solage-hsts.headers.stsIncludeSubdomains=true
6   - traefik.http.routers.solage.middlewares=solage-hsts

```

Listing 4.17 – Routage HTTPS et HSTS en production — `docker-compose.prod.yml`

À DÉVELOPPER — Procédure de déploiement (DEPLOYMENT.md)

Rédiger une procédure de déploiement documentée (critère C10) :

- **pré-requis** (serveur, Docker, variables d'environnement, nom de domaine, DNS) ;
- **étapes** de mise en production (build de l'image, `docker compose up`, migrations, vérification) ;
- procédure de **retour arrière** (*rollback*) en cas d'incident ;
- description des **environnements** (développement local, pré-production, production) et de leurs différences.

4.14 Contribution à la mise en production (DevOps)

Plusieurs briques DevOps sont déjà en place : conteneurisation complète (`docker compose`), images reproductibles, migrations **idempotentes** rejouables sans risque, et journalisation structurée adaptée à une collecte par l'orchestrateur. Il manque l'automatisation par intégration continue.

À DÉVELOPPER — Pipeline d'intégration continue (CI/CD)

Mettre en place un pipeline **GitHub Actions** déclenché à chaque *push*, et y joindre une capture d'un rapport d'exécution. Étapes cibles : analyse statique (PHPStan), tests (PHPUnit), *lint* JavaScript (ESLint), construction de l'image Docker. Squelette de départ à adapter :

```

1  name: CI
2  on: [push]
3  jobs:
4    build-and-test:
5      runs-on: ubuntu-latest
6      steps:
7        - uses: actions/checkout@v4
8        - uses: shivammathur/setup-php@v2
9          with: { php-version: '8.3' }
10       - run: composer install --no-interaction
11       - run: vendor/bin/phpstan analyse src modules      # analyse statique
12       - run: vendor/bin/phpunit                         # tests unitaires +
13         securite
14       - run: docker build -t solage-app .                 # construction du
15         livrable

```

Listing 4.18 – Squelette de pipeline CI proposé — `.github/workflows/ci.yml` (à implémenter)

4.15 Veille de sécurité

Sources suivies. La veille s'appuie sur l'OWASP (Top 10 et guides de test), les publications de l'ANSSI, les bulletins de sécurité de PHP et les bases de vulnérabilités (CVE).

Le déclic IDOR. Un article sur la fuite de données ayant touché l'URSSAF (accès aux données d'autres assurés en modifiant un identifiant dans l'URL — une faille **IDOR**) a déclenché un audit de SOLAGE. Il a révélé que les routes `/edituser/{id}`, `/api/users/delete` et `/api/posts/delete` vérifiaient l'*authentification* (`AuthMiddleware`) mais pas la *propriété* de la ressource : un utilisateur authentifié pouvait modifier ou supprimer le contenu d'autrui en changeant l'identifiant. Le correctif (contrôle « propriétaire ou administrateur » + 403 + journalisation) est décrit en section 4.7.

À DÉVELOPPER — Journal de veille

Compléter par un **journal de veille** daté : pour chaque entrée, la source, la date, ce qui a été appris, la vulnérabilité éventuellement détectée dans SOLAGE et le correctif apporté. Viser au moins deux à trois entrées en plus de l'IDOR.

Chapitre 5

Bilan, difficultés et perspectives

5.1 Satisfactions

Le projet a permis de construire et de *posséder* une application web multicouche complète : un socle « framework » écrit à la main (routeur, middlewares, logger PSR-3, utilitaires), une sécurité pensée couche par couche (XSS, CSRF, injection SQL, IDOR), une base PostgreSQL conçue proprement avec migrations idempotentes, et une chaîne Docker reproductible du développement à la production. La migration de MariaDB vers PostgreSQL et l’audit d’architecture (découplage des couches, suppression d’une requête N+1) ont particulièrement consolidé la compréhension du fonctionnement interne de l’application.

5.2 Difficultés rencontrées et résolues

Ces difficultés démontrent la **démarche structurée de résolution de problème** (compétence transversale T2) : chacune suit le schéma symptôme → diagnostic → correctif.

Constantes `STDOUT` / `STDERR` absentes en HTTP. Le logger écrivait via `fwrite(STDOUT, ...)` : les tests en ligne de commande passaient, mais toute requête HTTP renvoyait une erreur 500 (« *Undefined constant STDOUT* »). *Diagnostic* : ces constantes n’existent que dans le SAPI CLI de PHP. *Correctif* : utilisation des flux `php://stdout` / `php://stderr`, qui fonctionnent dans tous les SAPI. *Leçon* : tester en CLI ne garantit pas le bon fonctionnement en HTTP.

Headers already sent à la connexion. La réponse JSON était émise *avant* `session_start()`, empêchant la pose du cookie `PHPSESSID`. *Correctif* : démarrer la session dans le *bootstrap* (`public/index.php`), avant toute sortie possible.

Requête N+1 sur le fil. L’affichage des auteurs déclenchait une requête par message. *Correctif* : préchargement en une requête IN (section 4.8), avec gestion du cas IN () vide, interdit en PostgreSQL.

5.3 Perspectives

Les axes d’amélioration, déjà identifiés dans le dossier, sont :

- la **couverture de tests** automatisés (PHPUnit) et un pipeline d'**intégration continue** (chapitre 4) ;
- le **durcissement de la validation serveur** et du téléversement (contrôle MIME réel), ainsi que l'anti-fixation de session ;
- une passe d'**accessibilité** (RGAA) plus poussée et la factorisation des feuilles de style et scripts ;
- la résorption de la **dette de schéma** (table `responses` héritée *vs* mécanisme `reply_to`).

Annexe A

Annexes

Les annexes (40 pages maximum) rassemblent les éléments de la fonctionnalité la plus représentative : maquettes, captures et code, code des composants métier et d'accès aux données, code des autres composants, et jeux de tests complets.

A.1 Script de création de la base de données

Script complet `solage.pg.sql`, chargé à l'initialisation du conteneur PostgreSQL.

```
1 BEGIN;
2
3 CREATE TABLE roles (
4     id SERIAL PRIMARY KEY,
5     name VARCHAR(255) NOT NULL
6 );
7 INSERT INTO roles (id, name) VALUES (1, 'Admin'), (2, 'Utilisateur'), (3, '
    Modérateur');
8
9 CREATE TABLE users (
10     id SERIAL PRIMARY KEY,
11     name VARCHAR(255) NOT NULL,
12     firstname VARCHAR(255) NULL,
13     email VARCHAR(255) NOT NULL UNIQUE,
14     password VARCHAR(255) NOT NULL,
15     role INT NULL REFERENCES roles(id) ON DELETE SET NULL,
16     image VARCHAR(255) NULL
17 );
18 CREATE INDEX users_role_idx ON users(role);
19
20 CREATE TABLE posts (
21     id SERIAL PRIMARY KEY,
22     user_id INT NULL REFERENCES users(id) ON DELETE CASCADE,
23     date TIMESTAMP NOT NULL,
24     likes INT DEFAULT 0,
25     content TEXT NULL,
26     reply_to INT NULL,
27     reply_to_parent INT NULL,
28     image TEXT NULL
```

```

29 );
30 CREATE INDEX posts_user_idx ON posts(user_id);
31
32 CREATE TABLE responses (
33     id          SERIAL PRIMARY KEY,
34     post        INT NULL REFERENCES posts(id) ON DELETE CASCADE,
35     user_id     INT NULL REFERENCES users(id) ON DELETE CASCADE,
36     date        TIMESTAMP NOT NULL,
37     likes       INT DEFAULT 0,
38     content     TEXT NULL,
39     reply_to    INT NULL REFERENCES responses(id) ON DELETE SET NULL
40 );
41 CREATE INDEX responses_post_idx      ON responses(post);
42 CREATE INDEX responses_user_idx      ON responses(user_id);
43 CREATE INDEX responses_reply_to_idx  ON responses(reply_to);
44
45 CREATE TABLE likes (
46     id          SERIAL PRIMARY KEY,
47     user_id     INT NULL REFERENCES users(id) ON DELETE CASCADE,
48     post        INT NULL REFERENCES posts(id) ON DELETE CASCADE,
49     response    INT NULL REFERENCES responses(id) ON DELETE CASCADE,
50     created_at  TIMESTAMP NOT NULL
51 );
52 CREATE INDEX likes_user_idx          ON likes(user_id);
53 CREATE INDEX likes_post_idx          ON likes(post);
54 CREATE INDEX likes_response_idx      ON likes(response);
55
56 CREATE TABLE users_favorites_posts (
57     user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
58     post    INT NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
59     PRIMARY KEY (user_id, post)
60 );
61 CREATE INDEX users_favorites_posts_post_idx ON users_favorites_posts(post);
62
63 COMMIT;

```

A.2 Code d'un autre composant : le routeur

Routeur complet `src/Router.php` : correspondance d'URL, exécution des middlewares, armement du CSRF sur les requêtes POST, et instanciation du contrôleur.

```

1 class Router {
2     protected $routes = [];
3
4     public function addRoute($method, $path, $target, $middleware = null) {
5         $this->routes[] = ['method' => $method, 'path' => $path,
6                             'target' => $target, 'middleware' => $middleware
7                             ];
8     }

```

```

9      public function match() {
10          $requestUri      = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
11          $requestMethod = $_SERVER['REQUEST_METHOD'];
12
13          foreach ($this->routes as $route) {
14              $pathRegex = preg_replace('/\{([a-zA-Z0-9_]+\})/', '([a-zA-Z0-9_
15                  ]+)', $route['path']);
16              $pathRegex = '#^' . $pathRegex . '$#';
17
18              if ($route['method'] === $requestMethod && preg_match($pathRegex
19                  , $requestUri, $matches)) {
20                  if ($route['middleware']) {
21                      (new $route['middleware']())->handle();
22                  }
23                  if ($route['method'] === 'POST') {
24                      (new CsrMiddleware())->handle();
25                  }
26                  $routeArray = explode('#', $route['target']);
27                  if (count($routeArray) < 2) {
28                      throw new Exception("Le 'target' doit etre au format '
29                          Controller#Method'");
30                  }
31                  [$controller, $functionController] = $routeArray;
32                  if (isset($matches[1])) {
33                      $instance = new $controller($matches[1]);
34                      call_user_func_array([$instance, $functionController],
35                          $matches);
36                      return;
37                  }
38                  (new $controller())->$functionController();
39                  return;
40              }
41          }
42          http_response_code(404);
43          page404View::show();
44      }
45  }

```

A.3 Jeux de tests complets

À DÉVELOPPER — Jeux de tests (unitaires, intégration, sécurité)

Annexer ici l'intégralité des jeux de tests de la fonctionnalité la plus représentative, avec pour chaque test la **donnée d'entrée**, la **donnée attendue** et la **donnée obtenue** :

- code source des classes de test PHPUnit ;
- sortie complète de l'exécution (vendor/bin/phpunit) ;
- jeux de tests unitaires, d'intégration et de sécurité.

A.4 Maquettes et captures d'écran

À DÉVELOPPER — Maquettes et captures d'interfaces

Annexer :

- les **maquettes** (Figma) de l'ensemble des écrans ;
- les **captures d'écran** des interfaces réelles, mises en regard du code de la vue correspondante (au-delà de celles déjà présentées dans le corps du dossier).

Note. Le code source complet de SOLAGE (contrôleurs, modèles, vues, middlewares, configuration Docker) est disponible dans le dépôt Git du projet. Les extraits présentés dans ce dossier sont les plus significatifs au regard des compétences évaluées.